

Bachelor's Programme in Electronics and Electrical Engineering

Open-source floating-point square root algorithms

Max Laine

Author Max Laine

Title Open-source floating-point square root algorithms

Degree programme Electronics and Electrical Engineering

Major Electronics and Electrical Engineering

Teacher in charge D.Sc. Markus Turunen

Advisor M.Sc. Aleksi Korsman

Date 15 December 2024

Number of pages 37

Language English

Abstract

Floating-point arithmetic is a key aspect of modern computing, enabling precise real number calculations across a wide dynamic range. It is widely used in scientific computations, simulations, and computer graphics. Among the operations in floating-point arithmetic, square root calculations often have significant delays that impact performance. Open-source projects play a crucial role in the development of computational methods, offering innovative solutions for efficient floating-point arithmetic implementations.

This study presents a literature review of various square root algorithms, including a non-restoring iterative method, the CORDIC algorithm, lookup tables, and an algorithm using lookup tables and the Taylor series expansion for square roots. Additionally, the IEEE 754 standard for floating-point representation and associated rounding methods and exceptions, such as division by zero, are discussed.

The main focus is on three open-source floating-point unit (FPU) projects: FloPoCo, FPnew, and VFLOAT, which can be used on FPGA platforms. FloPoCo is also suitable for ASIC designs, while FPnew is tailored for RISC-V processors and systems-on-chip (SoCs). The results indicate that modern FPU projects prefer non-restoring iterative methods due to their speed and compact hardware requirements. FloPoCo is versatile and efficient for most applications, while FPnew offers varying precision for RISC-V implementations, particularly for machine learning. VFLOAT excels in high-precision applications.

Keywords square root, floating-point, open-source, FPGA, ASIC, RISC-V

Tekijä Max Laine

Työn nimi Liukulukujen avoimen lähdekoodin neliöjuurialgoritmit

Koulutusohjelma Elektroniikka ja sähkötekniikka

Pääaine Elektroniikka ja sähkötekniikka

Vastuupettaja Prof. Markus Turunen

Työn ohjaaja DI Aleksi Korsman

Päivämäärä 15.12.2024

Sivumäärä 37

Kieli englanti

Tiivistelmä

Liukulukuaritmetiikka on tärkeä osa modernia tietokonelaskentaa, sillä se mahdollistaa tarkat reaali-lukujen laskutoimitukset suurella dynaamisella alueella. Sitä hyödynnetään lukuisissa käyttökohteissa, laskennallisesta tieteestä simulaatioihin ja tietokonegrafiikkaan. Liukulukuaritmetiikan operaatioista neliöjuurella on usein merkittävä viive ja sen vaikutus suorituskykyyn on suuri. Avoimen lähdekoodin projektit ovat tärkeä osa laskentamenetelmien kehitystä, sillä eri projektit tarjoavat uusia ja innovatiivisia menetelmiä hyödyntää liukulukuaritmetiikkaa. Projektit tarjoavat useaan käyttötarkoitukseen sopivia neliöjuuritoteutuksia, jotka ovat nopeudeltaan, tehokkuudeltaan ja piirin kooltaan erilaisia.

Tämä työ on toteutettu kirjallisuustutkimuksena ja aineistoina on käytetty alan perusteoksia, tutkimusartikkeleita sekä muita aiheeseen liittyviä julkaisuja. Työssä tehdään kirjallisuuskatsaus erilaisiin neliöjuurialgoritmeihin. Nämä ovat ei-palauttava iteratiivinen menetelmä, CORDIC-algoritmi ja hakutaulut, sekä hakutauluihin ja neliöjuuren Taylor-sarjaan perustuva algoritmi. Lisäksi työ käsittelee IEEE 754 liukulukustandardia ja sen määrittämiä liukulukuformaatteja, pyöristämismenetelmiä ja poikkeuksia, kuten nollalla jakamista.

Tutkimuksen pääaiheena on kolme avoimen lähdekoodin liukulukuyksikkö projektia: FloPoCo, FPnew ja VFLOAT. Kaikkia projekteja voidaan soveltaa FPGA-alustalla, mutta FPnew on suunniteltu käytettäväksi RISC-V prosessoreille ja järjestelmäpiireille. FloPoCo soveltuu lisäksi ASIC-mikropiirille. Työn tuloksia voidaan hyödyntää sopivan avoimen lähdekoodin liukulukuyksikön valinnassa FPGA-, ASIC, tai RISC-V -projektiin.

Tulokset osoittavat, että modernit liukulukuyksikköprojektit suosivat ei-palauttavia iteratiivisia menetelmiä niiden nopeuden ja pienen laitteistokoon vuoksi. CORDIC-algoritmia hyödynnetään eniten trigonometristen funktioiden laskemiseen, eivätkä projektit käytä sitä neliöjuuren laskentaan. Hakutauluja ei käytetä suoraan neliöjuuren laskentaan niiden suuren muistivaatimuksen vuoksi, mutta niitä hyödynnetään usein muiden algoritmien osana. FloPoCo ja FPnew käyttävät ei-palauttavaa iteratiivista menetelmää, VFLOAT projekti käyttää hakutauluihin ja neliöjuuren Taylor-sarjaan perustuvaa algoritmia. Projektien vertailun perusteella voidaan todeta, että FloPoCo sopii hyvin lähes kaikkiin käyttökohteisiin sen suuren nopeuden, pienen laitteistokoon ja helppokäyttöisyyden vuoksi. FPnew tarjoaa FloPoCoon verrattavaa nopeutta pienemmällä laitteiston määrällä, ja mahdollisuuden vaihtelevaan liukuluvun tarkkuuteen operaatioiden välillä, joka parantaa tehokkuutta. Tämän vuoksi FPnew soveltuu parhaiten RISC-V toteutuksiin, jotka hyödyntävät muuttuvaa tarkkuutta operaatioiden välillä, esimerkiksi koneoppimiskäyttöön. VFLOAT-projektin neliöjuuritoteutuksen laitteistokoko on suuri, ja se on hitaampi kuin kahden muun projektin pienillä tarkkuuksilla. Projekti on nopeampi kuin FloPoCo suurilla liukulukujen tarkkuudella, joten se soveltuu korkeaa tarkkuutta vaativiin sovelluksiin joissa resurssien käyttö on joustavaa FPGA-alustalla.

Avainsanat neliöjuuri, liukuluku, avoin lähdekoodi, FPGA, ASIC, RISC-V

Contents

Abstract	2
Abstract (in Finnish)	3
Contents	4
Symbols & abbreviations	5
1 Introduction	6
2 IEEE 754 floating-point standard	7
2.1 Floating-point number representation	7
2.2 Rounding and precision	8
2.3 Exceptions	9
2.4 Basic operations	10
2.5 Square root operation	11
3 Floating-point square root algorithms	13
3.1 Non-restoring iterative algorithm	13
3.1.1 Integer square root algorithm	13
3.1.2 Floating-point square root algorithm	15
3.2 CORDIC	17
3.3 Lookup table	20
4 Open-source implementations of square root algorithms	23
4.1 FloPoCo	23
4.2 FPnew	28
4.3 VFLOAT	31
4.4 Comparison of projects	32
5 Conclusions	34
References	35

Symbols & abbreviations

Symbols

$(x)_2$	Value x in binary
$\bar{x}$	Boolean negation of x
$\lfloor x \rfloor$	x rounded down
$\lceil x \rceil$	x rounded up

Abbreviations

CORDIC	Coordinate Rotation Digital Computer
FPU	Floating-point unit
SoC	System-on-chip
RISC	Reduced instruction set computer
FPGA	Field-programmable gate array
ASIC	Application specific integrated circuit
VHDL	VHSIC Hardware Description Language
NaN	Not a Number
FMA	Fused multiply add

1 Introduction

Floating-point arithmetic is an essential part of modern computing as it can approximate a wide range of real values with high precision, making it common in numerous applications, from scientific simulations to graphics rendering [1]. Floating-point arithmetic provides a balance between speed and precision, making it flexible compared to the alternative, fixed-point arithmetic. It is particularly important for tasks involving complex calculations, where high numerical accuracy is required without sacrificing performance[1].

As development of floating-point units (FPUs) has focused primarily on improving the speed of multipliers and adders, operations like division and square root have seen less attention. While the square root operation is used much less than the others, accounting for only 0.33% of performed floating-point operations, the impact on performance is much greater than the other operations [2]. In many FPUs, the cycles taken by the square root operation can be tens of cycles more than the addition operation, which adds to the latency to computations. Floating-point operations are slower than fixed-point, so choosing an efficient, yet accurate algorithm for square root computation is important [2].

Open-source floating-point unit projects are a major part of integrated circuit design, as they allow for developers to easily test new designs for their own use cases, and make their own collaborations to the projects. In addition to this, open-source projects can be specialized in ways not possible for major commercial floating-point unit solutions, which provide solutions for the most common use case.

A literature review on floating-point square root algorithms found in textbooks and research papers is presented. The algorithms and methods covered are a non-restoring iterative algorithm, the CORDIC algorithm, lookup tables and an algorithm based on lookup tables and the Taylor series expansion for square roots. These algorithms all have differences in speed, efficiency and complexity, making them suitable for different use cases. The floating-point number formats, rounding, exceptions and basic operations defined by the IEEE 754 floating-point standard are also discussed.

This thesis presents a comprehensive review of open-source FPU-projects and the circuit-based techniques they use for floating-point square root computation. The results can be used in future hardware design when choosing a floating-point unit with a suitable method of square root computation in RISC-V-, FPGA- and ASIC-based projects. The projects covered are the FloPoCo project for FPGA- and ASIC-targeted floating-point operators with high user configurability, the FPnew floating-point unit, mainly used in RISC-V processors while also providing compatibility with FPGAs, and the VFLOAT project with a focus on variable precision computations for FPGAs.

Chapter 2 covers the 2019 revision of the IEEE 754 floating-point standard [3]. Chapter 3 comprises the theoretical section, discussing different hardware-based algorithms for square root computation. Chapter 4 focuses on real life open-source projects and the algorithms used by them. Chapter 5 presents the conclusions based on a comparison between the key factors, such as latency, frequency and resource usage of different projects, and suggests possible use cases for each project discussed.

2 IEEE 754 floating-point standard

Floating-point arithmetic is a method used by computers to handle real numbers in the form of finite precision decimal approximations. It is the most widely used method of representing real numbers in modern computers as it provides a good compromise between speed and accuracy, and is able to represent a wide range of values [4].

In order for results of floating-point operations to be consistent between computers, there must be a standardized way of representing and manipulating floating-point numbers. The first and most widely used standard is the IEEE 754 standard for floating-point arithmetic. The standard was released in 1985 after the development of the first floating-point coprocessor, the Intel 8087. Most modern processor instruction set architectures are compliant with IEEE 754, such as Intel's and AMD's x86_64 architectures, and the RISC-V architecture [5, 6, 7].

2.1 Floating-point number representation

In the IEEE 754-2019 [3] standard, a floating-point number is represented using the following parameters:

- b = radix, either 2 or 10
- p = precision i.e. number of digits in the mantissa
- $emax$ = maximum exponent
- $emin$ = minimum exponent

where $emin = 1 - emax$ for all formats. As this thesis focuses on hardware implementations, only binary formats, where $b = 2$, are considered. This is because circuit solutions rely heavily on bit shifts, which are correspondent to multiplying and dividing by 2.

Within each format, floating-point numbers are of the form

$$(-1)^s \cdot b^e \cdot m \quad (1)$$

where

- s is the sign, either 0 (positive) or 1 (negative)
- e is the exponent, any integer $emin \leq e \leq emax$
- m is the mantissa $0 \leq m < b$, of precision p

The exponent e is biased in order to represent both negative and positive exponents. This allows for the representation of very small and very large numbers. The encoded binary exponent is calculated by adding the bias to e . The bias in IEEE 754 is always equal to $emax$.

The mantissa has one digit (bit) before the radix point and $p - 1$ trailing bits composing the fractional part. The leading bit is not explicitly encoded, as it is determined by the exponent.

The parameters of common binary formats as defined by the standard are shown in table 1. It should be noted that binary16 is not a basic format, and a floating-point implementation can be complete without it included.

Table 1: Parameters of common binary formats

Parameter	binary16	binary32	binary64	binary128
k , storage width in bits	16	32	64	128
w , exponent width in bits	5	8	11	15
p , digits	11	24	53	113
t , trailing mantissa width in bits	10	23	52	112
e_{max}	+15	+127	+1023	+16383
$bias$	15	127	1023	16383

Each floating-point number has one encoding in IEEE 754-2019. In order to achieve this, numbers can be encoded as either normal or subnormal. Subnormal numbers allow for representation of numbers with a smaller magnitude than the formats minimum number, $b^{e_{min}}$.

A number is subnormal when the mantissa can be maximized such that $e = e_{min}$ and $0 < m < 1$, meaning there is no way of representing the mantissa without a leading 0. As an example, in binary32, the value $(-1)^0 \cdot b^{-126} \cdot 0.25$ would be subnormal, as $e = e_{min}$ and $0 < m < 1$. For subnormal numbers, the implicit leading bit of the mantissa is then always 0. With normal numbers, it is 1, and the mantissa is of magnitude $1 \leq m < b$. Subnormal numbers are encoded by setting all the bits of the exponent field to 0.

As an example, the binary32 radicand depicted in figure 1 will be examined. The most significant bit, the sign $s = 0$, makes the value positive. The exponent highlighted in gray is equal to 126 in decimal. The bias is included in this, so to get the actual exponent, it must be subtracted, resulting in $e = 126 - 127 = -1$. The trailing mantissa highlighted in green is equal to 0.25 in decimal. As the value is normal, the mantissa is $m = 1 + 0.25 = 1.25$ with the leading bit included. The value of the floating-point number can then be computed with formula 1 to be

$$(-1)^0 \cdot 2^{-1} \cdot 1.25 = 0.625 \quad (2)$$

Figure 1: Binary32 string



2.2 Rounding and precision

As the results of floating-point operations cannot be exactly represented, values must be rounded. IEEE 754-2019 [3] defines 4 rules for rounding in the binary formats; 1

for rounding to nearest and 3 directed roundings.

roundTiesToEven is the method for rounding to the nearest integer. If the value being rounded falls exactly between two values, it is rounded to the value whose least significant digit is even.

The 3 directed roundings include *roundTowardZero*, *roundTowardNegative* and *roundTowardPositive*. *roundTowardZero* rounds the value towards the value that is closest to zero, which is down for positive values and up for negative values. *roundTowardNegative* rounds towards negative infinity, effectively rounding down. *roundTowardPositive* rounds towards positive infinity, effectively rounding up.

The unit in last place (ulp) of a floating-point number refers to the gap between that number and the next representable floating-point number [4]. It depends on the precision and representation of the number in the system. Specifically, the ulp is equal to 2^{e-p+1} for normal numbers, and $2^{emin-p+1}$ for subnormals. The ulp reflects the smallest change that can be made to a number given the amount of bits available in the fractional mantissa. An error of 0.5 ulp - 1 ulp in means the result of the computation is accurate to the last bit. As an example, when computing a value to be equal to 1.000012, and the true exact result would be 1.000011, the difference is 1 ulp.

2.3 Exceptions

IEEE 754 [3] defines five kinds of exceptions which are signaled as a result of an operation. An exception being raised does not halt a sequence of operations by default, as it raises a flag which can be checked and corrected after the sequence is finished. These exceptions are

- *Invalid operation*: This exception is signaled when an operation receives an invalid input that cannot be computed, such as $\sqrt{-1}$, $0 \cdot \infty$ or $0/0$.
- *DivideByZero*: This exception is signaled if and only if an infinite result is defined for finite operands, such as $\log(0)$ or $1/0$. The default results are $\pm\infty$.
- *Overflow*: The overflow exception is signaled when the rounded result of an operation would have a larger exponent than $emax$ in the format of the operands. The default result of the operation is acquired by rounding the intermediate result. This is either $\pm\infty$ or the largest signed finite number of the format, with the sign determined by the sign of the intermediate result. The result depends on the rounding method being used.
- *Underflow*: The underflow exception is signaled when a result greater than 0 but smaller than b^{emin} is detected. This can be detected before or after rounding, as long as the method is consistent between operations. Handling of an underflow exception changes whether or not the result is exactly representable. By nature, subnormal numbers result in underflow, but as long as the result is exact, no flag is raised.
- *Inexact*: The inexact exception is signaled when the result of an operation cannot be represented exactly as a floating-point number, meaning the rounded and

unrounded results are not equal. [4] The result is a correctly rounded value. Default exception handling for overflow always results in inexact being signaled.

In addition to subnormal numbers, there are other reserved encodings for exponents and mantissas for representing special values. An exponent with all bits set to 1 can represent the values ∞ , $-\infty$ and NaN, "Not a Number". The infinities are acquired by setting all the bits in the mantissa to 0, not accounting for the implied leading bit. If not all bits in the mantissa are 0, then NaN is acquired. [3]

NaN is further split into two kinds, qNaN (quiet) and sNaN (signaling). The choice of NaN is determined by the performed operation, with qNaN being the default. NaNs are used to represent undefined or unrepresentable values, such as the result of an invalid operation. Quiet NaNs propagate through most floating-point operations without triggering exceptions, allowing for a program to continue execution. Signaling NaNs are used to signal the invalid operation exception when used in operations. It is used in cases where a variable is not defined properly such as in the case of an uninitialized variable. [4]

In IEEE 754, zero can be either positive or negative. The two zeros, +0 and -0 are encoded by setting all bits in the exponent and trailing mantissa to 0. This has certain benefits, such as in the case of taking the logarithm of x where x is a negative number that has underflowed to -0. Without a signed zero this computation, $\log(x) = \log(0)$ would result in $-\infty$ as the logarithm function in this case could not distinguish between an underflowed negative value and a zero. With the underflowed $x = -0$, the logarithm function knows that an underflow has occurred for a negative number and returns a qNaN. [8]

2.4 Basic operations

IEEE 754 [3] defines numerous operations which can broadly be classed into computational operations and non-computational operations. Computational operations involve the computation of floating-point numbers, and in either NaN, floating-point or integer results. These may or may not signal exceptions, depending on the operation. Non-computational operations do not produce floating-point results, and they do not signal exceptions.

Operations can be further classified based on the operands and results of the operation. Homogenous operations require that operands and results are all of the same format. For example, if a homogenous operation is performed on a binary32 number, the result is also a binary32 number. *formatOf* operations indicate the format of the result, which is independent of the result. Due to the scope of this thesis, it will mainly be discussing the *formatOf* general-computational basic arithmetic operations.

These *formatOf* basic arithmetic operations defined by IEEE 754 [3] are **addition**, **subtraction**, **multiplication**, **division**, **squareRoot** and **fusedMultiplyAdd**.

The result of **squareRoot**(x) has a positive sign for all values of $x \geq 0$, except $\sqrt{-0}$ having negative -0 as the result. **fusedMultiplyAdd**(x, y, x) computes $(x \cdot y) + z$. The main difference to using the multiplication and addition operations separately is that fused multiply add (FMA) only requires one rounding as opposed to two, one after

multiplication and one after addition, leading to greater accuracy [4]. This operation is important in software implementations for division and square root operations, as the Goldschmidt algorithm, which relies heavily on FMA, can be used to compute these operations [9].

Operations can result in all exceptions, including in overflow and underflow [4]. In order to avoid this, results of operations are normalized by reducing or increasing the exponent so that the mantissa of the number meets the defined requirements, $1 \leq m < b$ for normal numbers and $0 < m < 1$ for subnormal numbers, where $b = 2$ for binary formats [4]. For example, suppose the addition operation on two binary values $x = (1.0000)_2$ and $y = (1.1111)_2$. The result of this computation would be $x + y = (10.1111)_2$ in binary, or 2.9375 in decimal. In this case, the number has overflowed, as $2.9375 > b$. This number would then be normalized by moving the decimal place in the binary representation until the leading bit is 1, resulting in $(1.01111)_2$, with the amount of places moved left being added to the exponent [4]. Conversely, if the decimal place had been moved right, the amount would be subtracted from the exponent. This gives $x + y$ the floating-point number representation $(-1)^0 + 2^1 \cdot (1.01111)_2$. Figure 2 displays whether the result of an operation returns NaN, is rounded or is normalized

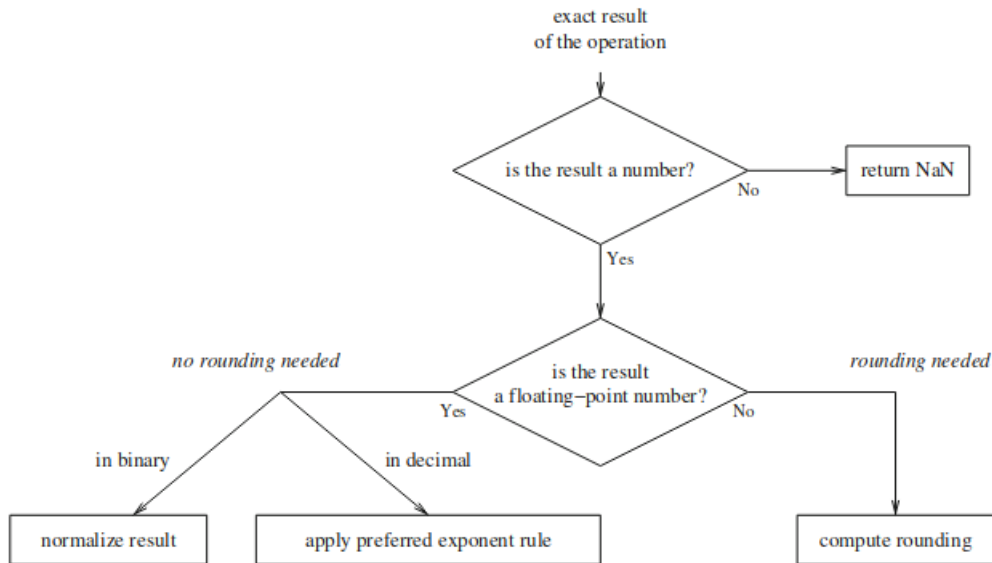


Figure 2: Specification of the implementation of a floating-point operation. Adapted from [4], p. 234

2.5 Square root operation

IEEE 754 [3] does not define an equation or algorithm to be used for computation of square roots or any other arithmetic operations discussed in the previous section. The standard however does require that the result is correctly rounded according to the specific rounding mode used, whether it is to nearest, towards zero, or toward positive/negative. The steps of computing the square root of a normalized non-negative

binary32 are as follows:

1. Split the mantissa and exponent parts of the 32-bit radicand.
2. Compute the square roots of both parts individually. In order to compute the square root of the exponent, the mantissa must be shifted either 1 or 0 bits left so that the exponent becomes even [10]. For example, the value $2^{-65} \cdot (1.0111)_2$ would have one subtracted from the exponent and the mantissa shifted left one time, making the new value equal to $2^{-66} \cdot (10.111)_2$. The square root of the exponent then becomes $e' = \frac{e}{2}$ for an even original exponent e , and $e' = \frac{e-1}{2}$ for an odd original exponent.
3. The square root of the shifted mantissa $\sqrt{m'}$ is then calculated using an algorithm covered in the following sections.

For radix-2, the shifted mantissa of a normal number always lies within $1 \leq m' < 4$, so the result of the square root operation will be normal [10]. For subnormal numbers no shifting of the mantissa is needed as the exponent is always divisible by 2 in each format of IEEE 754 [3]. However, the square root of a subnormal number must be normalized after it is computed [11].

Some algorithms, such as CORDIC [12], require that the mantissa is normalized in the range $[0.5, 2)$. For normal numbers with even exponents, no bit shifts are needed as the mantissa is always in the range $[1, 2)$ by definition. In the case of odd exponents, the mantissa would be shifted one bit right, halving it and making it fit the range $[0.5, 1)$. The new exponent would then be $e' = \frac{e+1}{2}$.

3 Floating-point square root algorithms

This section examines different square root algorithms found in literature, such as the non-restoring iterative method covered in [11], CORDIC, lookup tables, and a lookup table and Taylor series based algorithm. Some performance characteristics, such as iterations to compute a square root, will be covered shortly in this chapter. More in depth characteristics such as latencies, frequencies and resource usage will be covered in later chapters.

3.1 Non-restoring iterative algorithm

This section covers the non-restoring square root implementation covered in the conference proceedings [11, 13]. The non-restoring iterative square root algorithm is a low-cost method of computing a square root, using only a single adder/subtractor to compute the square root of the mantissa of a floating-point number. "Non-restoring" means that the partial remainder, R , computed with each iteration is used in the next iteration, regardless of whether it is negative or positive. A restoring algorithm would restore the state of the partial remainder on negative values [11]. The algorithm generates an exact bit in each iteration, meaning it is accurate to the last bit of the number [11].

3.1.1 Integer square root algorithm

In order to understand the floating-point square root algorithm, it is helpful to first be familiar with the working principle of the integer square root algorithm covered by [11].

Assuming a 32-bit unsigned binary radicand, denoted by $D = D_{31}D_{30}D_{29}\dots D_1D_0$, with D_k being the bit at position k . Each pair of bits in the radicand can be mapped to a single bit in the resulting square root Q , which means the square root is at most 16 bits in size, $Q = Q_{15}Q_{14}\dots Q_1Q_0$. The remainder $R = D - Q \cdot Q$ has at most 17 bits.

The algorithm for computing the square root of a 32-bit integer is then

1. Initialize $q_{16} = 0$ and $r_{16} = 0$, where $q_k = Q_{15}Q_{14}\dots Q_k$ has $16 - k$ bits, and $r_k = R_{16}R_{15}\dots R_k$ has $17 - k$ bits (for example, $q_{14} = Q_{15}Q_{14}$).
2. Iterate for k from 15 to 0.
3. If $r_{k+1} \geq 0$, $r_k = r_{k+1}D_{2k+1}D_{2k} - q_{k+1}01$
else $r_k = r_{k+1}D_{2k+1}D_{2k} + q_{k+1}01$
4. If $r_k \geq 0$, $q_k = q_{k+1}1$ else $q_k = q_{k+1}0$
5. Repeat steps 3 and 4 until $k = 0$. If $r_0 < 0$, $r_0 = r_0 + q_01$

This algorithm computes one bit of the result Q with each iteration, which means it takes 16 clock cycles in total to compute the square root of a 32-bit value. In general,

one pair of bits in the input number is processed with each iteration, so the iterations required are half of the width of the input binary value.

The non-restoring iterative algorithm uses the two's complement representation for computation, which means the most significant bit in intermediate results determines the sign. This is dropped when the value is used in calculations. For example, if $r_3 = 100$, it would be used as 00 in the calculation of $r_2 = r_3 D_7 D_6 = 00 D_7 D_6$.

Terms such as $r_{k+1} D_{2k+1} D_{2k}$, $q_{k+1} 1$ denote concatenation of bits, not multiplication. These can be achieved by bit shifting the terms and summing them together. $r_{k+1} D_{2k+1} D_{2k}$, $q_{k+1} 1$ would then be computed as $(r_{k+1} \ll 2) + (D_{2k+1} \ll 1) + (D_{2k})$, where $\ll i$ is a left shift by i bits.

As an example for q_k with Q of length 4, suppose $Q = Q_3 Q_2 Q_1 Q_0 = (1010)_2$. This would make q_k have $4 - k$ bits, making $q_4 = 0$ by default. On a positive r_3 in the next iteration, q_3 would then be equal to $q_3 = q_4 1$, or $q_3 = (0 \ll 1) + 1 = 01$.

Figure 3 depicts the circuit for computing the 16-bit square root of a 32-bit integer. A 32-bit radicand is placed into register D , and is shifted left twice with each iteration. The square root result is stored in register Q , and is shifted left once with each iteration. Register R , split into two $R0, R2$, contains the partial remainder. The control input, determined by the value of $R2$ (item 3 in the breakdown of the algorithm), controls the mode of the adder/subtractor. When the control input is 0 it subtracts and when it is 1 it adds. The three gates after D (OR, XNOR and INV respectively) are used to compute $D_{2k+1} D_{2k} - 01$ or $D_{2k+1} D_{2k} + 11$. This iterative design only required one adder/subtractor to compute the square root. This circuit is a part of the floating-point implementation, used for calculating the square root of the mantissa.

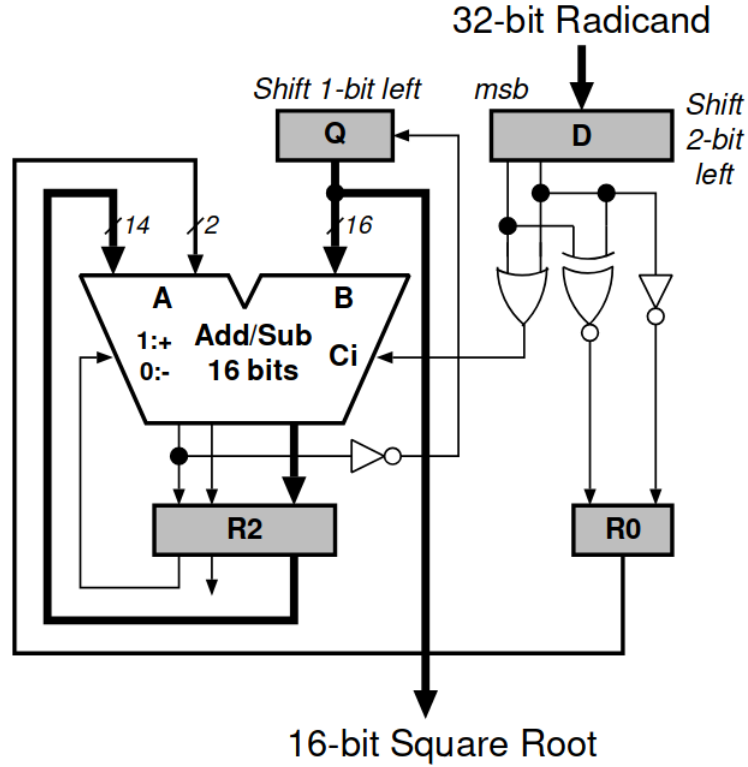


Figure 3: 16-bit square root of 32-bit integer. Adapted from [11], p. 228

3.1.2 Floating-point square root algorithm

The floating-point non-restoring iterative square root algorithm covered in [11] works by splitting the original floating-point value into the sign, exponent and mantissa. The exponent for the square root is calculated with an adder with the equation $\frac{(E-127)}{2} + 127 = \frac{E}{2} + 63 + E\%2$, where E is the biased exponent (e.g. $0 < E < 255$ for binary32), and $\%2$ is the modulo of 2 operation, computing the remainder of $E/2$. This way of computing the exponent, and by extension shifting of the mantissa, is the same as the first method covered in Section 2.5. If the mantissa is shifted left (odd exponent), it is of the form $1x.xxx...xx$, and if it is not shifted (even exponent), it is of the form $01.xxx...xx$, where the prepended zero is required for the computation of the square root [14]. As the result Q has one bit for every pair of bits in D , the decimal point is always after the first bit in Q , making the result of the form $Q = 1.yyy...yy$. This makes the length of the mantissa equal to $p + 2$, where the $+2$ is due to both the hidden bit and the shift or prepended zero, which would be 25 bits for binary32 floating-point numbers. Figure 4 illustrates a floating-point square root unit for binary32 values. The algorithm requires 24 clock cycles to compute the square root of a 24-bit mantissa.

The square root of the mantissa is computed with the same circuit as an integer square root. In order to make the presented circuit usable for floating-point numbers, the mantissa has zeroes appended until the length is doubled, so for a single precision mantissa this means appending zeroes until the length is 48 bits. The result of this

computation is normalized by default for normal number inputs as covered in Section 2.5, with the shifted mantissa being in the range $[1, 3)$ thus resulting in the square root of the mantissa being in the range $[1, 2)$. If the input is a subnormal number, it must be normalized after the computation of the square root.

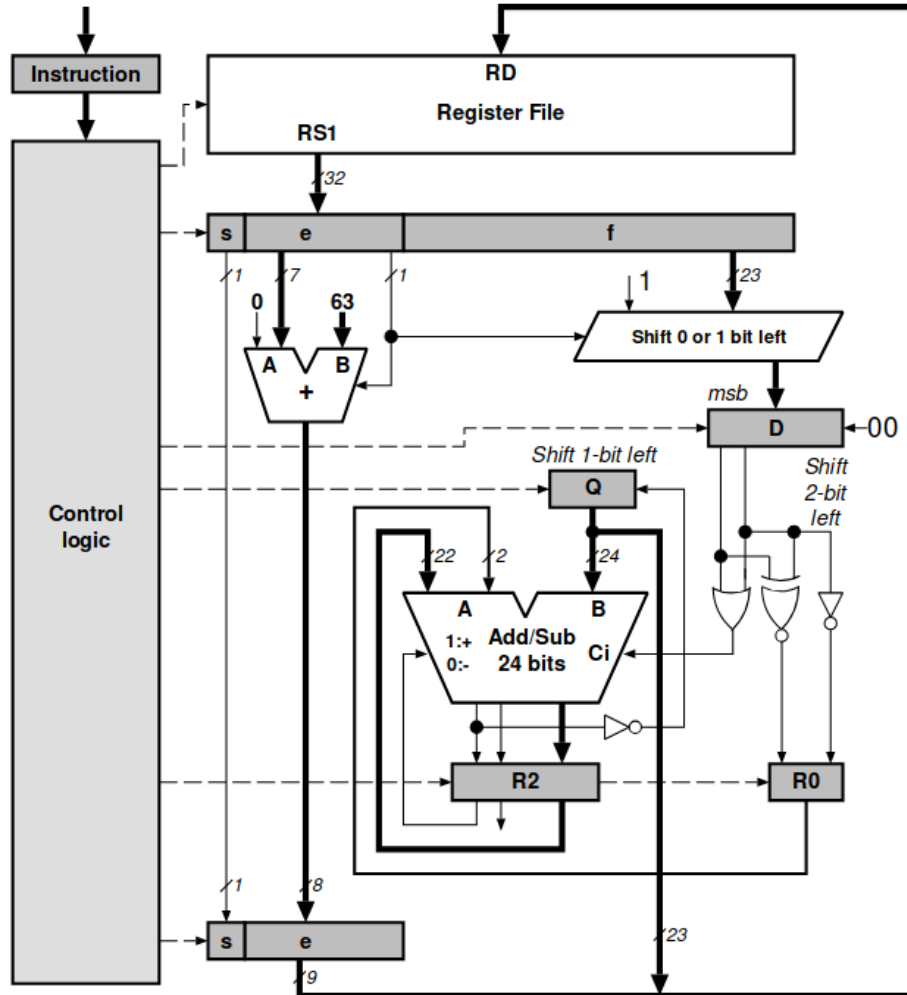


Figure 4: 24-bit square root of 24-bit floating-point mantissa. Adapted from [11], p. 230

As an example, suppose the floating-point number 1.0. For brevity, an 8-bit floating-point number, with 1 bit for sign, 4 bits for exponent and 3 bits for fractional mantissa, will be used for square-root computation. This floating-point number would have a bias of 7. The 8-bit radicand would then be $(0_0111_000)_2$ for 1.0, with the mantissa including the hidden 1 being $(1.000)_2$. In order to compute the square root of this 4-bit mantissa, its length must be doubled to be 8-bits long. This means prepending 1 zero and appending 3 zeroes. The value is then loaded into register D, and the square root Q can be computed using the algorithm presented for integer square roots. The intermediate result of Q will be shown after each q_k computation.

$$D = D_7 D_6 \dots D_0 = 01.0000000$$

Set $q_4 = 0$ and $r_4 = 0$. Iterate from $k = 3$ to $k = 0$.

$$k = 3$$

$$r_4 \geq 0, r_3 = r_4 D_7 D_6 - q_4 01 = 001 - 001 = 000$$

$$r_3 \geq 0, q_3 = q_4 1 = 01 \quad (Q_3 = 1, Q = 1)$$

$$k = 2$$

$$r_3 \geq 0, r_2 = r_3 D_5 D_4 - q_3 01 = 0000 - 0101 = 1011$$

$$r_2 < 0, q_2 = q_3 0 = 010 \quad (Q_2 = 0, Q = 1.0)$$

$$k = 1$$

$$r_2 < 0, r_1 = r_2 D_3 D_2 + q_2 11 = 01100 + 01011 = 10111$$

$$r_1 < 0, q_1 = q_2 0 = 0100 \quad (Q_1 = 0, Q = 1.00)$$

$$k = 0$$

$$r_1 < 0, r_0 = r_1 D_1 D_0 + q_1 11 = 011100 + 010011 = 101111$$

$$r_0 < 0, q_0 = q_1 0 = 01000 \quad (Q_0 = 0, Q = 1.000)$$

$$\text{As } r_0 < 0, r_0 = r_0 + q_0 1 = 1011111 + 010001 = 1000000 = (-)0$$

This gives the result $Q = 1.000$ and $R = 000000$, the remainder being unsigned.

3.2 CORDIC

CORDIC (COordinate Rotation DIgital Computer) is an algorithm used for the calculation of tasks such as trigonometric, logarithmic and hyperbolic functions, multiplication, exponentials, division and square roots [15]. The algorithm is derived from the general rotation transform in two dimensions, where a vector with end point (x_i, y_i) is rotated by α_i , resulting in the new end point of (x_{i+1}, y_{i+1}) [16].

$$\begin{bmatrix} x_{i+1} \\ y_{i+1} \end{bmatrix} = \begin{bmatrix} \cos(\alpha_i) & -\sin(\alpha_i) \\ \sin(\alpha_i) & \cos(\alpha_i) \end{bmatrix} \begin{bmatrix} x_i \\ y_i \end{bmatrix} \quad (3)$$

With the new coordinates (x_{i+1}, y_{i+1}) being

$$x_{i+1} = x_i \cdot \cos(\alpha_i) - y_i \cdot \sin(\alpha_i) \quad (4)$$

$$y_{i+1} = x_i \cdot \sin(\alpha_i) + y_i \cdot \cos(\alpha_i) \quad (5)$$

Or alternatively

$$x_{i+1} = \cos(\alpha_i)(x_i - y_i \cdot \tan(\alpha_i)) \quad (6)$$

$$y_{i+1} = \cos(\alpha_i)(y_i + x_i \cdot \tan(\alpha_i)) \quad (7)$$

The rotation angles are then restricted so that $\tan(\alpha_i) = \pm 2^{-i}$. This way the multiplication by the tangent term becomes a bit shift [17]. By replacing the decision of whether or not to rotate with the decision of whether to rotate clockwise or counterclockwise with each iteration i , the equations can be further simplified by reducing the cosine terms to a constant d_i [17]. The iterative rotations can then be expressed with the equations

$$x_{i+1} = K_i(x_i - y_i \cdot d_i \cdot 2^{-i}) \quad (8)$$

$$y_{i+1} = K_i(y_i + x_i \cdot d_i \cdot 2^{-i}) \quad (9)$$

where

$$K_i = \cos(\tan^{-1} 2^{-i}) = \frac{1}{\sqrt{1 + 2^{-2i}}}$$

$$d_i = \pm 1$$

The scale factor K_i can be removed by moving it to the side of the result of the equation in order to make the iterative equations result in a simple shift and add algorithm [17]. The factor is caused by CORDIC using pseudo-rotations instead of true rotations, which means each rotation affects the length of the vector [18]. This is illustrated in figure 5, where E^{i+1} is the end point of the vector E^i after a pseudo-rotation (with scaled length R^{i+1}). This factor on the results side, K_n , can be precomputed, and is equal to $K_n = \prod_{n=1}^{\infty} \sqrt{1 + 2^{-2n}} \approx 1.646760258121$ [16].

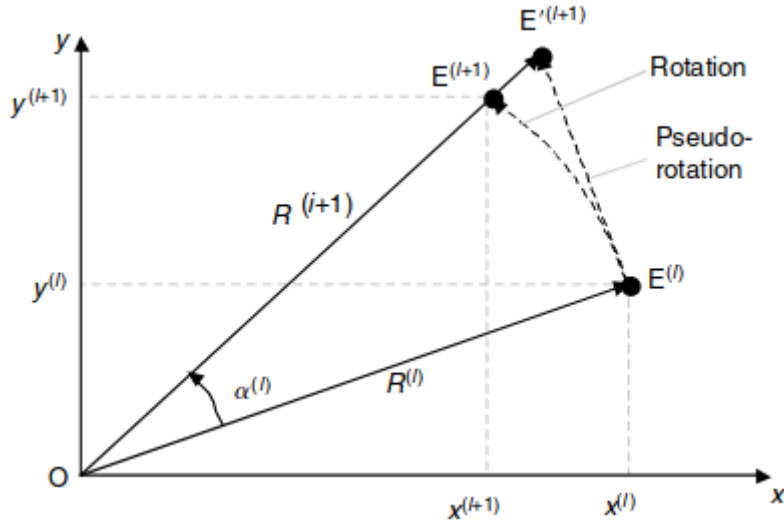


Figure 5: A pseudo-rotation step in CORDIC. Adapted from [18], p. 460.

In order to calculate trigonometric, linear and hyperbolic modes, the generalized CORDIC algorithm includes a mode variable m , which corresponds to the three different modes (circular, linear and hyperbolic) by taking the values -1 for hyperbolic, 0 for linear, or 1 for circular [17]. The generalized CORDIC equations are

$$x_{i+1} = x_i - m \cdot y_i \cdot d_i \cdot 2^{-i} \quad (10)$$

$$y_{i+1} = y_i + \cdot x_i \cdot d_i \cdot 2^{-i} \quad (11)$$

$$z_{i+1} = z_i - d_i \cdot e_i \quad (12)$$

z_i is the angle accumulator, which stores the total rotations performed. e_i is the elementary angle of rotation used in the selected mode, taking values $\tan^{-1}(2^{-i})$ for circular ($m = 1$), 2^{-i} for linear ($m = 0$), and $\tanh^{-1}(2^{-i})$ for hyperbolic ($m = -1$) [17]. As the scale factor of the iterative equations K_i for circular mode was derived from $K_i = \cos(\tan^{-1} 2^{-i}) = \frac{1}{\sqrt{1+2^{-2i}}}$, for hyperbolic mode it is $K_i = \cosh(\tanh^{-1} 2^{-i}) = \frac{1}{\sqrt{1-2^{-2i}}}$. This then makes $K_n = \prod_{i=1}^{\infty} \sqrt{1-2^{-2i}} \approx 0.8281593609602$ for hyperbolic mode. The iterations $i = 4, 13, 40, \dots, i, 3i + 1$ must be computed twice in order to reach convergence, making the product $\prod_{1,2,3,4,4,5,\dots}$ [17].

CORDIC can be used in 2 operation modes, rotation and vectoring mode. In rotation mode, the angle accumulator is initialized with the desired angle. The goal of rotation mode is to apply small rotations to the vector to converge on the desired angle [17]. In vectoring mode, the rotator rotates the vector in order to align the vector with the x-axis, making y equal to zero with enough iterations [17].

As square root is a hyperbolic function, it is computed using CORDIC in hyperbolic vectoring mode. This mode provides the following results for the iterative equations as the number of iterations n approaches $+\infty$ [12]:

- $x_n \approx K_n \sqrt{x_0^2 - y_0^2}$
- $y_n \approx 0$
- $z_n \approx z_0 + \tanh^{-1}(\frac{y_0}{x_0})$

For square root computation only the x_n result is needed. The square root of value v can then be computed by selecting $x_0 = v + 0.25$ and $y_0 = v - 0.25$ for the initial values [12]. The acquired equation is

$$\begin{aligned} x_n &= K_n \sqrt{x_0^2 - y_0^2} \\ &= K_n \sqrt{(v + 0.25)^2 - (v - 0.25)^2} \\ &= K_n \sqrt{(v^2 + 0.5v + 0.0625) - (v^2 - 0.5v + 0.0625)} \\ &= K_n \sqrt{(v^2 + 0.5v + 0.0625 - v^2 + 0.5v - 0.0625)} \\ &= K_n \sqrt{v} \end{aligned} \quad (13)$$

As the CORDIC gain K_n can be precomputed, it can be removed using a suitable multiplier (equal to $\frac{1}{K_n} \approx 1.20749706776$) in a hardware implementation. The algorithm requires k iterations for k bits of precision. For example, the computation of the square root of the 24-bit mantissa of a single precision floating-point number takes 24 iterations [18].

3.3 Lookup table

A lookup table (LUT) is a method of storing precomputed values in a table, allowing for fast function approximation by replacing runtime computation with a retrieval from an array of data. Storing all values of a mantissa and its square root would be expensive for large formats. For example, a simple table storing approximations of 24-bit single precision mantissa square roots would require a table of size $2^{24} \times 24$ bits, which results in a memory size of approximately 50.33 megabytes [19]. Tables for storing approximates in literature commonly store approximates with accuracy of 14 bits or fewer, such as the one used for acquiring an approximation for the Newton-Raphson algorithm in [20]. This implies a lookup table used for direct computation of a half precision IEEE 754 floating-point number, which has a precision of 11 bits, is reasonable.

As the required memory size of a table for direct square root approximation becomes very large as the amount of accurately approximated bits increases, table-based methods have been developed to take advantage of the high speed of lookup tables as well as the lower hardware cost of algorithmic techniques. One of these methods is an algorithm using a combination of the Taylor series expansion for square roots and a small lookup table. This method is covered by [21, 22, 23].

The architecture of this algorithm is depicted in 6. Here Y refers to the value of the IEEE 754 mantissa, with a value in the range $[1, 2)$.

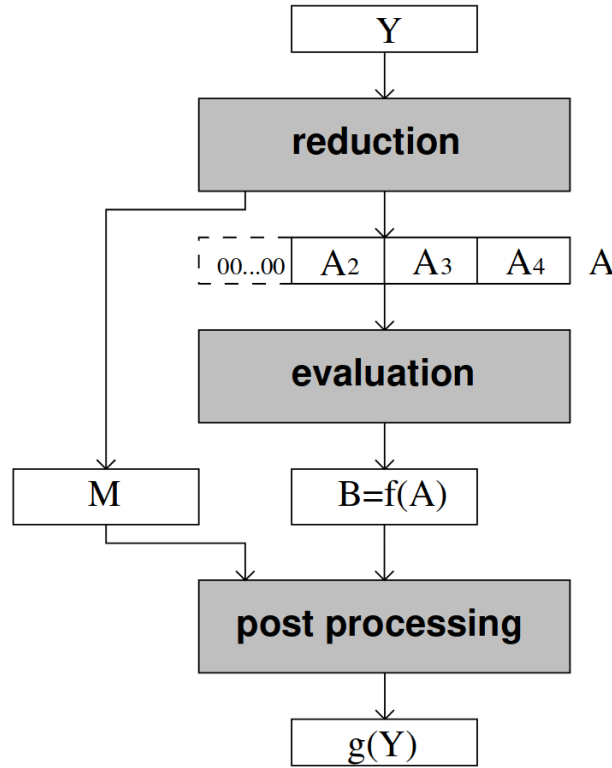


Figure 6: Architecture of Taylor series expansion and table-based function approximation. Adapted from [22], p. 7.

The reduction step, depicted in figure 7 starts with taking a truncation of the mantissa Y to k bits. This truncation is denoted with $Y^{(k)}$, where k is equal to $\lfloor (p+2)/4 \rfloor + 1$, which is 7 for single precision and 14 for double precision values [21]. This k -bit value is used to index into a lookup table to provide an approximation $R = \frac{1}{Y^{(k)}}$, the reciprocal of the k -bit approximation. R has $k+1$ accurate bits. A value A used for the Taylor series evaluation is acquired using this value, where $A = Y \times R - 1$. A has a reduced range of $-2^{-k} < A < 2^k$. Another value used for post-processing, M , is evaluated using a second lookup table. This value, also indexed with $Y^{(k)}$, provides a $4k$ -bit approximation of $\frac{1}{\sqrt{R}}$. This value for M is used if the exponent of the original value is even. If it is odd, another table is used which provides the value $M = \frac{\sqrt{2}}{\sqrt{R}}$. Exponent calculation is the same as the first method covered in Section 2.5, and this additional coefficient of $\sqrt{2}$ accounts for the left shift of the mantissa. A is split into four k -bit sections, with the first section being only zeroes. A then takes the form

$$A = A_2 z^2 + A_3 z^3 + A_4 z^4 \quad (14)$$

where $z = 2^{-k}$ and $|A_i| \leq 2^k - 1$.

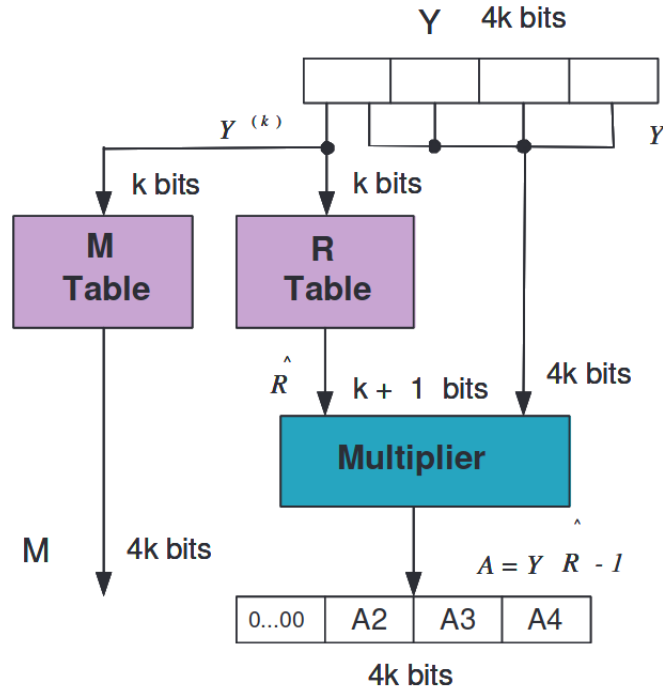


Figure 7: Architecture of reduction step in lookup table based square root. Adapted from [23], p. 39.

In the evaluation step, the goal is to compute the approximation $B = \sqrt{1 + A}$ using the Taylor series expansion

$$B = \sqrt{1 + A} \approx 1 + \frac{A}{2} - \frac{1}{8}A^2z^4 - \frac{1}{4}A_2A_3z^5 + \frac{1}{16}A_2^3A_2z^6 \quad (15)$$

This requires three multipliers, of which two are square ($k \times k$ bits) for A_2^2 and A_2A_3 , and one is rectangular ($2k \times k$ bits) for $A_2^2 \times A_2$. The architecture of the evaluation step is depicted in figure 8.

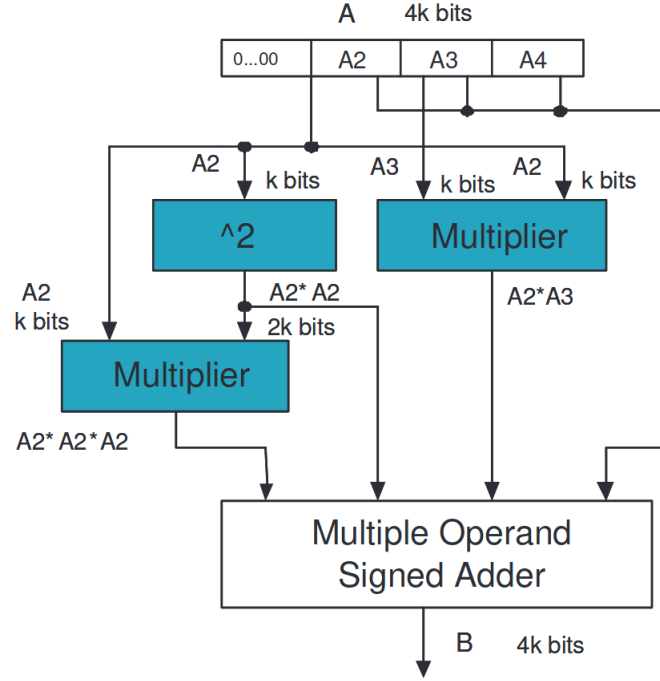


Figure 8: Architecture of Taylor series evaluation step. Adapted from [23], p. 40.

In the post-processing step, only correction of the result B is performed. This is accomplished by calculating a multiplication to acquire the result $\sqrt{Y} = B \times M$. According to [23], the overall error of all performed computations in total is analyzed to be 2.39×2^{-4k} , meaning the algorithm is accurate to the last bit. As this algorithm is not iterative, only a single pass through the steps from Y to $\sqrt{Y}$ is needed. The performance of this algorithm will be discussed further in a later chapter.

4 Open-source implementations of square root algorithms

This chapter covers three open-source floating-point unit solutions, namely FloPoCo as described in [24], FPnew [25], and VFLOAT [21]. While numerous open-source floating-point unit projects exist, FPnew, FloPoCo and VFLOAT are among the most used and maintained projects with backing from large universities. These projects are competitive even when compared to vendor solutions from Xilinx and Altera, two of the leading FPGA manufacturers. The solutions from Xilinx and Altera are not covered in this thesis, however the results of the comparisons with FloPoCo and VFLOAT can be found in [21].

4.1 FloPoCo

FloPoCo (Floating-Point Cores) [24] is an open-source project focused on generation of arithmetic cores for FPGAs. FloPoCo generates application-specific operators using user-defined parameters for specifications such as input format, output format (not applicable to square root) and operator-specific parameters. The input parameters also include target FPGA, target frequency and operator-specific controls. Target FPGA by default is the AMD Kintex7 and target frequency is 0Hz. Operator-specific controls include parameters such as number of test counts with a test bench, or whether to output a file of inputs and outputs for an operator or not. FloPoCo's architecture generator outputs synthesizable VHDL (VHSIC Hardware Description Language) based on the input parameters.

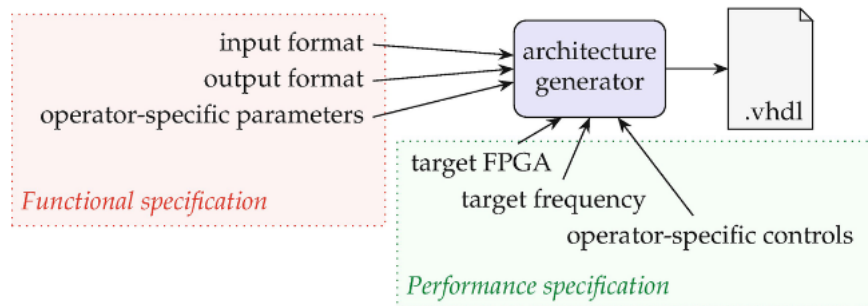


Figure 9: Basic structure of the FloPoCo architecture generator. Adapted from [24], Section A2.2.

FloPoCo will automatically attempt pipelining operators to the specified target frequency. Pipelining is a method of allowing an operator to function at a higher frequency by splitting the operator into several stages, allowing for parts of operations to be completed individually of each other by using intermediate registers between the stages. Using pipelining, a new input can be accepted and a new output returned with each clock cycle. FloPoCo calculates the pipeline depth based on the operator used and the target frequency. A pipeline with e.g. depth 3 has 4 distinct stages that the

operator circuit is divided into with registers between each stage. This is illustrated in figure 10.

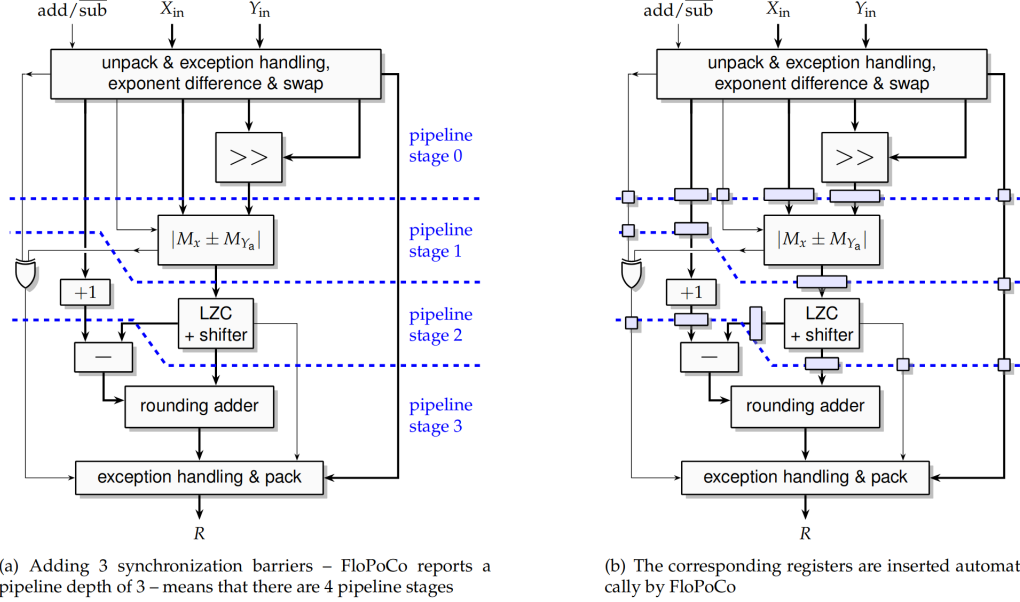


Figure 10: Pipelining of a FloPoCo floating-point adder. Adapted from [24], Section A2.5.

FloPoCo supports, but is not limited to, IEEE 754 binary floating-point number formats.

The square root operator in the most recent FloPoCo version 5.0 takes three input parameters; exponent size in bits wE , fractional mantissa size in bits wF and an optional method parameter. Method set to 0 uses a plain restoring square root algorithm, and 1 uses a non-restoring algorithm. This value defaults to 1 if not specified. An IEEE 754 single precision square root operator would be created with the terminal command `flopoco FPSqrt wE=8 wF=23`. This produces a file called `flopoco.vhdl` containing VHDL for the generated operator.

FloPoCo is not truly IEEE 754 compliant. In addition to allowing for non-IEEE 754 formats to be created by the user, the FloPoCo floating-point number format, called "Nfloat", uses two additional bits before the sign bit to encode special values that would be encoded in the mantissa and exponent fields in IEEE 754 formats. These encodings are described in table 2.

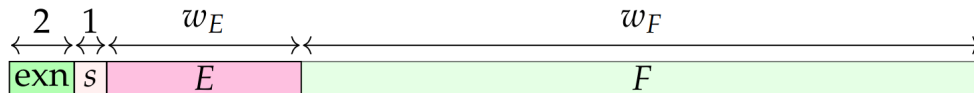


Figure 11: FloPoCo Nfloat floating-point number format. Adapted from [24], Section 2.5.3.

exn	encoding
00	zero, signed as in IEEE 754
01	normal numbers
10	signed infinity
11	NaN

Table 2: Encodings of the FloPoCo Nfloat format

FloPoCo is capable of handling subnormal numbers for square roots. As the square root of a number will never output a subnormal number, only inputs must be managed. Without encoding subnormal numbers in the way IEEE 754 defines, FloPoCo achieves this by using an intermediate format where subnormal numbers are normalized with an exponent one bit wider than wE . As the exponent is halved in computing the square root, the resulting exponent will be small enough to fit the original wE . This normalization involves a leading zero counter and one shifter.

As mentioned, the two methods for floating-point square root computation in FloPoCo are a non-restoring algorithm, similar to the one as discussed in Section 3.1, and a restoring algorithm. These are both classified as digit-recurrence algorithms, meaning they form one digit of the result with each iteration [20]. Due to this and the additional guard bit for rounding, IEEE 754 numbers take $p + 1$ iterations to compute a result using FPSqrt. Both methods inputs are bound to the range $[1, 4)$, and outputs $[1, 2)$.

Both digit-recurrence algorithms, which are covered in detail in [24], are based on the general digit by digit formulation for radix 2

$$S_i = \sum_{j=0}^i 2^{-j} d_{-j} \quad (16)$$

where

- S_i : Increasingly accurate result of square root computation.
- d_{-i} : Digit of the function being evaluated in base 2. Negative $-i$ here means d_{-i} starts from most significant and ends in the least significant bits.

In binary arithmetic, where the base $b = 2$, each digit d_{-i} can only be either 1 or 0. The restoring algorithm, with one step depicted in figure 12, takes advantage of this by assuming $d_{-i} = 1$, then attempting the subtraction $2W_i - 2S_i + 2^{-i-1}$ and testing whether the sign is negative or positive. If the result of the subtraction is negative, then the correct value to select was $d_{-i} = 0$. This is what makes the algorithm restoring, as a way to "restore" the remainder if a subtraction fails exists. W_i here refers to the remainder at iteration i , and T_i is called the "tentative next remainder", defining the value of d_{-i-1} .

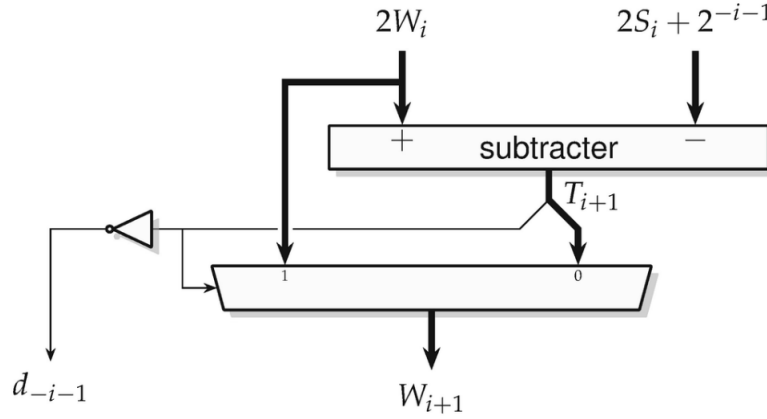


Figure 12: Architecture of one step of restoring iterative square root. Adapted from [24], Section 19.1.3.

The algorithm for one step is defined by the following equations:

$$W_0 = M - 1 \quad (17)$$

$$T_{i+1} = 2W_i - (2S_i + 2^{-i-1}) \quad (18)$$

$$d_{-i-1} = \begin{cases} 1 & \text{if } T_{i+1} \geq 0 \\ 0 & \text{if } T_{i+1} \leq 0 \end{cases} \quad (19)$$

$$W_{i+1} = \begin{cases} T_{i+1} & \text{if } T_{i+1} \geq 0 \\ 2W_i & \text{if } T_{i+1} \leq 0 \end{cases} \quad (20)$$

Where M is the value of the square root operand. S_i is acquired from the recursive formula

$$S_i = S_{i-1} + 2^{-i} d_{-i} \quad (21)$$

This operation and $2S_i + 2^{-i-1}$ do not require addition hardware, as the addition can be implemented as a bit concatenation. As S_i is calculated to $p + 1$ precision, S_i is computed from S_0 to S_{24} for single precision values. This last bit is then used as the round bit. If the extra bit is equal to 1 the result is rounded up and if it is equal to 0 it is truncated. First iterations are faster than last ones due to iteration i requiring a subtractor on only $i + 3$ bits.

The architecture for one step of the non-restoring algorithm is depicted in figure 13. "ufix(m, l)" and "sfix(m, l)" denote an unsigned and signed fixed-point format with bits from the most significant bit m to the least significant bit l . In short, a negative value means the MSB/LSB is to the right of the decimal point, and a positive would mean it is on the left. For example, the unsigned fixed-point number $(111.1111)_2$ is in the format ufix(2, -4), the unsigned number $(0.0111)_2$ is in the format ufix(-2, -4), and the signed number (1.0111) is in the format sfix(0, -4). The fractional mantissa interpreted as a fixed-point number is in the ufix(-1, - wF), as it is in the range $[0, 1)$. The fractional mantissa for a single precision number would then have the format ufix(-1, -23).

Instead of W_i , the non-restoring algorithm uses T_i as the current remainder. The following equations define one step of the non-restoring algorithm:

$$S_{-1} = 0 \quad (22)$$

$$T_0 = M - 1 \quad (23)$$

$$d_{-i} = \begin{cases} 1 & \text{if } T_i \geq 0 \\ 0 & \text{if } T_i \leq 0 \end{cases} \quad (24)$$

$$S_i = S_{i-1} + 2^{-i} d_{-i} \quad (25)$$

$$U_i = 2S_i + 2^{-i} \overline{d_{-i}} + 2^{-i-1} \quad (26)$$

$$T_{i+1} = \begin{cases} 2T_i + U_i & \text{if } d_{-i} = 0 \\ 2T_i - U_i & \text{if } d_{-i} = 1 \end{cases} \quad (27)$$

Here $\overline{d_{-i}}$ is the boolean negation of d_{-i} , i.e. it has the opposite value of the original d_{-i} . All additions except for $2T_i \pm U_i$ are bit concatenations, so the only hardware required for the algorithm is an inverter for d_i and an adder/subtractor. As shown in the figure 13, sign information for T_i is stored and is kept through the addition/subtraction. According to [24], this algorithm maps better to modern FPGAs LUT (lookup table) and fast carry logic, which refers to how carry propagation and routing is performed on modern FPGAs. This results in the method having lower area and critical path, which determines the maximum delay. In practice, the non-restoring algorithm results in less latency than the restoring algorithm, which is why it is the default method used in FloPoCo.

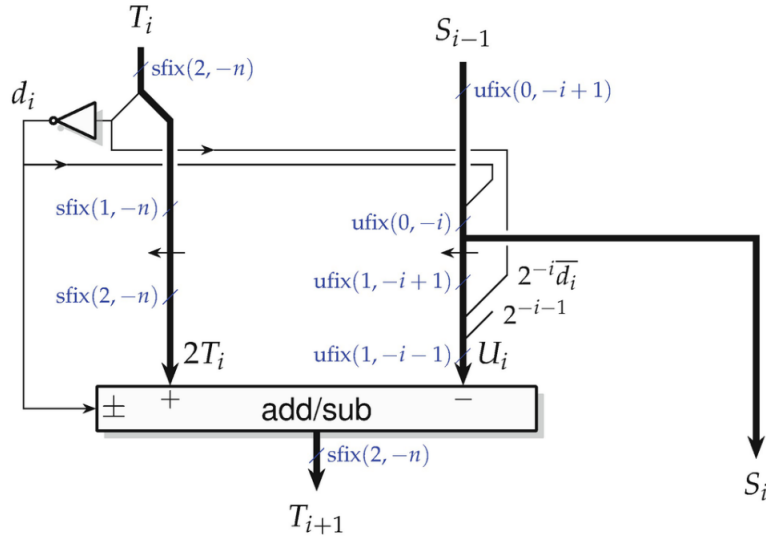


Figure 13: Architecture of one step of non-restoring iterative square root. Adapted from [24], Section 19.1.4.

FloPoCo has previously included a piecewise polynomial approximation method for computing square roots, called `FPSqrtPoly`, which has been removed since version 4.2 of FloPoCo. This method, which works by splitting the input range into sub-intervals, approximating each with precomputed polynomials of fixed degree, and evaluating them efficiently using fixed-point arithmetic and Horner’s method, is described in more detail in [20].

In addition to the floating-point square root algorithms, FloPoCo includes methods to compute the square root of a fixed-point number. One of these methods involves using CORDIC to compute the 2D norm of x and y . This function is called `Fix2DNorm` in FloPoCo, and it computes the operation $\sqrt{x^2 + y^2}$.

4.2 FPnew

FPnew [25] is an open-source high configurable transprecision floating-point unit (TP-FPU) generator. It is mainly designed in use for RISC-V processors and systems-on-chip (SoCs), but includes tools for target-specific synthesis for FPGAs and ASICs. As opposed to FloPoCo, FPnew offers true IEEE 754 compliance for its standard formats.

In traditional computing systems guaranteed and consistent precision for each computation step has been the standard for years [26]. This leads to Transprecision computing is a paradigm of enabling high control over precision in computation in order to achieve high energy efficiency without sacrificing performance or quality of results. This is accomplished by only using the format needed without excess precision for each step of computation. The graph in figure 14 depicts the potential energy efficiency gain from a transprecision computation method over a traditional one. For mixed-precision workloads, FPnew can reach a speedup of 1.67 with respect to a standard binary32 baseline, along with reducing energy consumption by 37% [25]. This makes it optimal for applications such as neural network training, and big data analytics [26].

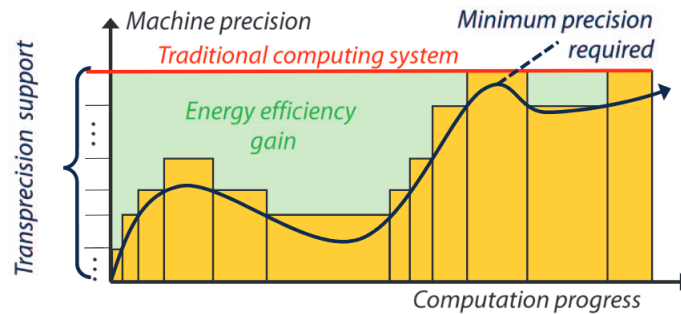


Figure 14: Transprecision computation compared to a traditional computing system. Adapted from [26], p. 1106.

The available binary floating-point number formats in FPnew are as follows:

- **binary64:** IEEE 754 double precision (11, 52)

- **binary32**: IEEE 754 single precision (8, 23)
- **binary16**: IEEE 754 half precision (5, 10)
- **binary16alt**: Custom half precision (8, 7)
- **binary8**: Custom quarter precision (5, 2)

Values in parentheses denote mantissa width and exponent width respectively. The format *binary16alt* has been popularized under the name *bfloat16*, an alternative floating-point number format developed by Google. The difference to *bfloat16* is that *binary16alt* follows IEEE 754 principles regarding normal and subnormal numbers, NaNs, zeroes, infinities and rounding. The *binary16alt* format has the same dynamic range but lower precision than a single precision floating-point number, leading to higher performance and energy efficiency while using less space in memory, making it suitable for applications where high precision is not required. The *binary8* and *binary16* formats have the same relationship.

An FPnew TP-FPU is grouped by operations. The 4 operation groups are

- *ADDMUL*: addition, multiplication and FMA operations
- *DIVSQRT*: division and square root operations
- *COMP*: comparisons and bit manipulations
- *CONV*: conversions between floating-point formats, as well as conversions between floating-point number and integer

The top level view of the FPU is depicted in figure 15. Each of the individual groups or the entire TP-FPU can be enabled or disabled by clock gating, a power-saving technique that disables the clock signal to inactive parts of a circuit.

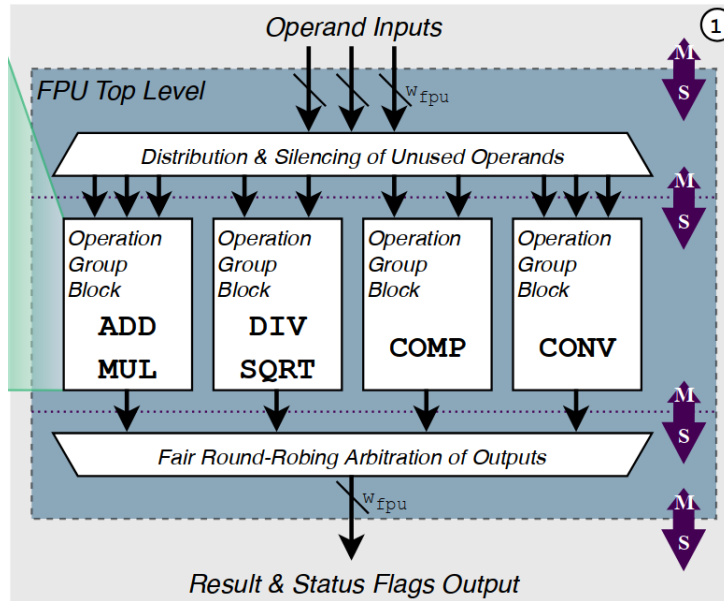


Figure 15: FPU top level. Adapted from [25], p. 4.

The square root and division algorithms used by FPnew are both non-restoring iterative algorithms, with square root being implemented using the algorithm discussed in Section 3.1. Implementation of these algorithms for binary32 values is covered in [27]. FPnew includes three different DIVSQRT blocks, with one of them only being for binary32 valued inputs. The architecture of both algorithms is shared, with a module called "nrbd_nrsc_tp" being responsible for computing the square root of the mantissa.

Subnormal numbers are supported as inputs, with a prenormalization involving a leading zero counter to count the amount of bits to shift the mantissa left, and subtract from the exponent. Exponent calculation is handled using the first method covered in Section 2.5, including for normalized subnormal values. A parameter called "Precision_ctl_SI" can be used to control the precision of the output to have fewer bits of precision than the input. This can be used to achieve lower latency at the cost of lower accuracy.

An iteration of the non-restoring square root/division is unrolled three times, which means each cycle computes three iterations and thus creates three new bits for the result [28]. Loop unrolling here is used as a method of increasing throughput and reducing latency by creating multiple hardware blocks to compute multiple iterations during one clock cycle. The square root of a single precision number, regardless of output precision, is computed in 8 cycles. Using "Precision_ctl_SI" to overwrite the precision to be lower than that of the input allows also reduces the clock cycles required to compute the square root. The latencies related to different output sizes is depicted in table 3. One cycle is used for pre-processing, which includes decomposition of the input to its sign, mantissa and exponent parts, detection of subnormal numbers using leading zero detectors, and computing the exponent. The iteration stage of mantissa calculation using the non-restoring iterative algorithm takes 3-6 cycles, depending on the size of the output precision. One final cycle is used for post-processing, which involves normalization, rounding, and composition of the resulting value.

Precision_ctl_SI	Latency (clock cycles)
8-11	5
12-15	6
16-19	7
20-23	8

Table 3: Latency with respect to output mantissa width.

Latency results are synthesized on hardware using UMC 65 nm process technology. UMC 65 nm refers to a 65-nanometer semiconductor manufacturing process technology developed by United Microelectronics Corporation (UMC), used to fabricate integrated circuits.

4.3 VFLOAT

VFLOAT [21] is a floating-point unit library with a focus on variable precision, meaning widths of the mantissa and exponent fields can be configured for generated FPUs to allow for additional formats, similar to FloPoCo. This allows the FPU to be tailored to high- and low-precision tasks to optimize the amount of resources used. This project only supports FPGAs unlike the other projects discussed.

While VFLOAT is based on the IEEE 754-2008 standard, some features are not included. VFLOAT does not support subnormal numbers, and only two rounding modes are used: `roundTowardZero` and `roundTiesToEven`. The additional formats by VFLOAT, which can have any exponent and mantissa fields from 2 bits upward, are not standard IEEE 754 formats. These formats still follow the basic sign - exponent - mantissa -structure.

The components of VFLOAT are designed to be used in a pipelined manner. The architecture of each component supports this, as depicted in figure 16. Input port OP(s) signify the input operands. These are provided from the previous component(s) in the pipeline through the RESULT output port(s). Output port DONE is responsible for signaling to the next component that the operation is done, and the next operation in the pipeline can start, using the input port READY. Exceptions are propagated through the pipeline. EXCEPTION_OUT tells whether an exception has occurred, and EXCEPTION_IN receives this information.

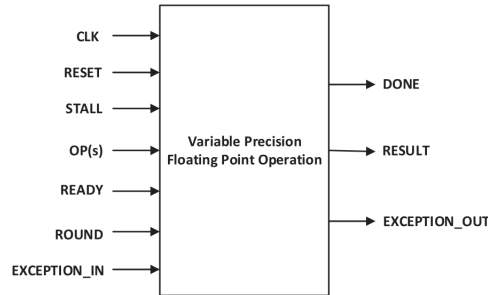


Figure 16: Generic VFLOAT component. Adapted from [21], p. 6.

The square root algorithm used by VFLOAT is the Taylor series- and table-based algorithm covered in Section 3.3. In previous versions, the square root algorithm used by VFLOAT had a precision of about 2.39 ulp, which made it too inaccurate to be considered for high-precision applications. However, in the most recent versions of VFLOAT the error has been evaluated to have been smaller than 1 ulp.

As opposed to FloPoCo, which allows the user to set a target frequency and creates a pipeline automatically suited to that frequency, VFLOAT allows the user to select the number of clock cycles/pipeline depth of the operators. The total latency of the square root operation is determined by the latencies of the multipliers, with the formula

$$YR + 2S + M + L + 2 \quad (28)$$

where YR, S, M and L are the latencies of their respective multipliers. The minimum latency is 7 clock cycles, when all multiplier latencies are equal to 1. A number of factors affect these latencies, such as frequency and pipeline depth. The architecture of the VFLOAT square root is depicted in figure 17.

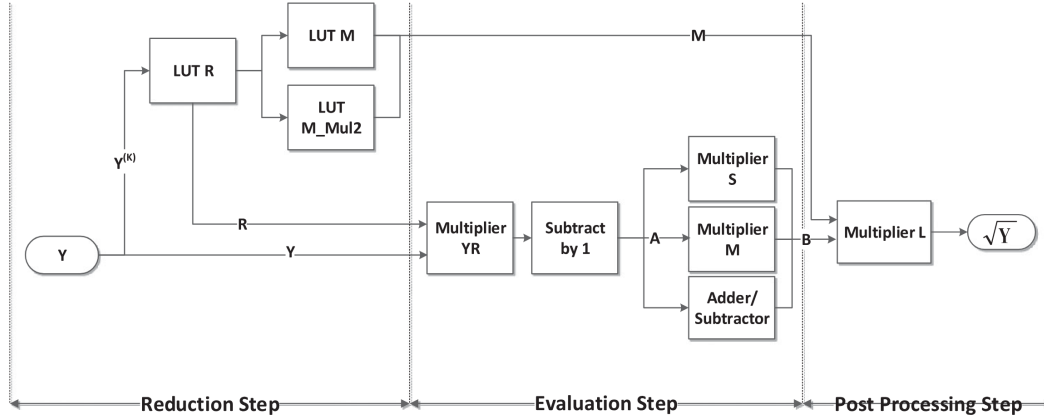


Figure 17: VFLOAT square root architecture. Adapted from [21], p. 9.

The LUTs for the square root algorithm are provided as memory initialization files for both Xilinx and Altera FPGAs. VFLOAT in its most recent version provides tables from $k = 2$ to $k = 14$, where $\lfloor (p + 2)/4 \rfloor + 1$, as covered in Chapter 3.3. This means the largest mantissa (including the hidden bit) that has lookup tables provided for it is 53 bits wide, which is the mantissa size of a double precision floating-point number.

VFLOAT always performs one iteration for an accurate square root result due to the nature of the algorithm. However, due to the small lookup tables and large amount of multipliers used, the hardware cost of the VFLOAT square root is high.

4.4 Comparison of projects

Table 4 presents a comparison of the latencies and maximum frequencies for single precision values or their closest equivalent between the different implementations discussed in this work. Values for UMC 65 nm FPnew and FloPoCo are provided by [29]. The rest of the values are by [21].

Values for FloPoCo and VFLOAT on the Virtex 6 FPGA are based on an extended format with exponent width of 9 and mantissa width of 30. Single precision values for the two FPGA-based implementations were not found in the cited articles or other up-to-date literature, but these values provide a close approximate of frequencies and latencies. Values for FPGAs contain approximations, indicated by inequality signs. Note that cycles for VFLOAT are chosen by user, but a lower latency for single precision values may be acquired using more or less cycles.

	FPnew	FloPoCo	FloPoCo (FPGA)	VFLOAT
Latency (clock cycles)	8	6	≤ 8	11
Frequency (MHz)	350	350	≥ 172	≥ 191
Latency (ns)	23	17	≤ 47	≤ 57
Testing platform	UMC 65nm	UMC 65nm	Virtex 6	Virtex 6
Maximum error (ulp)	≤ 1 [26]	< 1 [24]	< 1 [24]	< 1 [21]

Table 4: Performance comparison of open-source floating-point square root algorithms for single precision floating-point numbers.

In addition to the values in table 4, area comparisons for FloPoCo and FPnew are compared in [29]. In terms of "kilo gate equivalents" (kGE), a metric representing the area of a circuit relative to a 2-input NAND gate, the FloPoCo design, using the UMC 65 nm process, has an area of 19kGE, while FPnew has 7.6kGE, approximately 40% of the area used by FloPoCo. The resource comparisons for FPGA-based FloPoCo and VFLOAT from [21] indicate that VFLOAT has a higher hardware cost than FloPoCo. Due to the multipliers used by VFLOAT, the gate equivalent would be much higher than that of FloPoCo. Hardware usage between these two are compared in table 5. The resources compared are registers (Reg), lookup tables (Slice LUTs), digital signal processing logic elements (DSP) and block RAMs (BRAM). These are logic blocks used in FPGAs.

	FloPoCo (FPGA)	VFLOAT
Resources	335 Reg, 684 Slice LUTs	528 Reg, 776 Slice LUTs, 8 DSP, 3 BRAM

Table 5: FloPoCo and VFLOAT resource usage on Virtex 6 FPGA

In double precision (binary64) comparisons, VFLOAT is faster than FloPoCo. Due to the similarities, it can be assumed that it would be faster than FPnew as well, but no data is available for double precision square root computations for FPnew. This comparison, along with resource usage, is depicted in table 6. Values are provided by [21].

	FloPoCo (FPGA)	VFLOAT
Latency (clock cycles)	7	7
Frequency (MHz)	91	145
Latency (ns)	77	48
Testing platform	Virtex 6	Virtex 6
Resources	489 Reg, 1,741 Slice LUTs	1,546 Reg, 6,243 Slice LUTs, 58 BRAM

Table 6: FloPoCo and VFLOAT double precision comparison

5 Conclusions

Current open-source FPU generators and libraries have a focus on computing with only the precision needed, increasing performance and efficiency in comparison to traditional computing methods by not using computation time and resources on unnecessary bits of precision. FloPoCo and VFLOAT achieve this by allowing for creation of variable precision components, with exact exponent and mantissa widths provided by the user. In addition to offering multiple standard and non-standard floating-point number formats, the transprecision approach chosen by FPnew allows for input and output sizes of square root computations to differ, reducing latency and increasing energy efficiency.

Modern open-source floating-point unit implementations in FloPoCo and FPnew favor a digit-recurrence based method, namely a non-restoring iterative algorithm, as these have very low hardware usage while maintaining faster convergence and low latency. In FPGA use cases where more hardware is available, the VFLOAT project can provide a competitive table-based method for square root computation. CORDIC-based algorithms are rarely used for square root computation, with the open-source project FloPoCo using it mainly for fixed-point trigonometric computations.

In conclusion, when comparing the projects, FloPoCo is seemingly the fastest for single-precision computations, while FPnew offers the best efficiency and smallest area on chip in addition to transprecision support, with similar speed to FloPoCo. VFLOAT scales well for larger bit widths, providing the lowest latency for double precision computations, but at a high hardware cost. This makes it best for applications where high and precision are critical. However, it is poorly suited to general applications due to being highly expensive and slow for more common floating-point formats. FloPoCo, with high configurability and ease of use, is well suited to most FPGA- and ASIC-based applications, regardless of precision. This is due to it providing the fastest square root implementation for most precisions, in addition to low hardware cost. FloPoCo has the added benefit of having a simple command-line interface for fast creation of synthesizable VHDL. FPnew should be considered in applications where energy efficiency and area of the FPU are important constraints, such as low-power RISC-V projects. FPnew also excels in mixed-precision applications involving, e.g., machine learning or big data analytics.

References

- [1] V. Patil, A. Raveendran, P. M. Sobha, A. David Selvakumar, and D. Vivian, “Out of Order Floating Point Coprocessor for RISC-V ISA,” in *2015 19th International Symposium on VLSI Design and Test*, 2015, pp. 1–7.
- [2] S. Oberman and M. Flynn, “Design issues in division and other floating-point operations,” *IEEE Transactions on Computers*, vol. 46, no. 2, pp. 154–161, 1997.
- [3] “Ieee standard for floating-point arithmetic,” *IEEE Std 754-2019 (Revision of IEEE 754-2008)*, pp. 1–84, 2019.
- [4] J.-M. Muller *et al.*, *Handbook of Floating-Point Arithmetic*, 2nd ed. Birkhäuser Boston, 2018.
- [5] M. Cornea, “Ieee 754-2008 decimal floating-point for Intel® architecture processors,” in *2009 19th IEEE Symposium on Computer Arithmetic*, 2009, pp. 225–228.
- [6] *AMD64 Architecture Programmer’s Manual*, 2024. [Online]. Available: <https://www.amd.com/content/dam/amd/en/documents/processor-tech-docs/programmer-references/40332.pdf>
- [7] A. Waterman and K. Asanović, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, 2017. [Online]. Available: <https://riscv.org/wp-content/uploads/2017/05/riscv-spec-v2.2.pdf>
- [8] D. Goldberg, “What every computer scientist should know about floating-point arithmetic,” *ACM Comput. Surv.*, vol. 23, no. 1, p. 5–48, 1991. [Online]. Available: <https://doi.org/10.1145/103162.103163>
- [9] P. Markstein, “Software division and square root using Goldschmidt’s algorithms,” in *Proceedings of the 6th Conference on Real Numbers and Computers (RNC’6)*, vol. 123. Citeseer, 2004, pp. 146–157.
- [10] M. Vladutiu, *Computer arithmetic: algorithms and hardware implementations*. Berlin: Springer, 2012.
- [11] Y. Li and W. Chu, “Implementation of single precision floating point square root on fpgas,” in *Proceedings of The 5th Annual IEEE Symposium on Field-Programmable Custom Computing Machines (Cat. No.97TB100186)*, 1997, pp. 226–232.
- [12] MathWorks, “Compute Square Root Using CORDIC,” <https://se.mathworks.com/help/fixedpoint/ug/compute-square-root-using-cordic.html>, accessed: 18-10-2024.

- [13] Y. Li and W. Chu, "A new non-restoring square root algorithm and its vlsi implementations," in *Proceedings International Conference on Computer Design. VLSI in Computers and Processors*, 1996, pp. 538–544.
- [14] Y. Li, "Question about "Implementation of Single Precision Floating Point Square Root on FPGAs"." Email correspondence, 2024, received: 2024-10-28.
- [15] P. K. Meher, J. Valls, T.-B. Juang, K. Sridharan, and K. Maharatna, "50 years of cordic: Algorithms, architectures, and applications," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 56, no. 9, pp. 1893–1907, 2009.
- [16] B. Yang, D. Wang, and L. Liu, "Complex division and square-root using cordic," in *2012 2nd International Conference on Consumer Electronics, Communications and Networks (CECNet)*, 2012, pp. 2464–2468.
- [17] R. Andracka, "A survey of CORDIC algorithms for FPGA based computers," in *Proceedings of the 1998 ACM/SIGDA Sixth International Symposium on Field Programmable Gate Arrays*, ser. FPGA '98. New York, NY, USA: Association for Computing Machinery, 1998, p. 191–200. [Online]. Available: <https://doi-org.libproxy.aalto.fi/10.1145/275107.275139>
- [18] P. Behrooz, "Computer arithmetic: Algorithms and hardware designs," *Oxford University Press*, vol. 19, p. 490, 2000.
- [19] M. Schulte and J. Stine, "Approximating elementary functions with symmetric bipartite tables," *IEEE Transactions on Computers*, vol. 48, no. 8, pp. 842–847, 1999.
- [20] F. de Dinechin, M. M. Joldes, B. Pasca, and G. Revy, "Multiplicative square root algorithms for FPGAs," in *International Conference on Field Programmable Logic and Applications*. Milano, Italy: IEEE, Aug. 2010, p. 14. [Online]. Available: <https://ens-lyon.hal.science/ensl-00475779>
- [21] X. Fang and M. Leeser, "Open-source variable-precision floating-point library for major commercial fpgas," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 9, no. 3, Jul. 2016. [Online]. Available: <https://doi.org/10.1145/2851507>
- [22] M. Ercegovic, T. Lang, J.-M. Muller, and A. Tisserand, "Reciprocation, Square root, Inverse Square Root, and some Elementary Functions using Small Multipliers," no. LIP RR-1997-47, p. 24, Nov. 1997. [Online]. Available: <https://hal-lara.archives-ouvertes.fr/hal-02101940>
- [23] X. Wang, "Variable Precision Floating-Point Divide and Square Root for Efficient FPGA Implementation of Image and Signal Processing Algorithms," PhD thesis, Northeastern University, Boston, Massachusetts, Dec. 2007.
- [24] F. de Dinechin and M. Kumm, *Application-Specific Arithmetic*. Springer, 2024. [Online]. Available: <https://link.springer.com/book/10.1007/978-3-031-42808-1>

- [25] S. Mach, F. Schuiki, F. Zaruba, and L. Benini, “Fpnew: An open-source multi-format floating-point unit architecture for energy-proportional transprecision computing,” p. 14, 2020. [Online]. Available: <https://arxiv.org/abs/2007.01530>
- [26] A. C. I. Malossi, M. Schaffner, A. Molnos, L. Gammaitoni, G. Tagliavini, A. Emerson, A. Tomás, D. S. Nikolopoulos, E. Flamand, and N. Wehn, “The transprecision computing paradigm: Concept, design, and applications,” in *2018 Design, Automation & Test in Europe Conference & Exhibition*, 2018, pp. 1105–1110.
- [27] L. Li, *Datasheet of FP Divisor and Square Root for Variable Precision*, https://github.com/pulp-platform/fpu_div_sqrt_mvp/blob/86e1f558b3c95e91577c41b2fc452c86b04e85ac/document/Datasheet_of_transprecision.pdf, 2017, accessed: 2024-10-21.
- [28] S. Mach, “Floating-point architectures for energy-efficient transprecision computing,” Doctoral Thesis, ETH Zurich, Zurich, 2021.
- [29] L. Li, M. Gautschi, and L. Benini, “Approximate div and sqrt instructions for the risc-v isa: An efficiency vs. accuracy analysis,” in *2017 27th International Symposium on Power and Timing Modeling, Optimization and Simulation (PATMOS)*, 2017, pp. 1–8.